

1. Overview

UVSim is a simple virtual machine that helps computer science students learn how machine language and basic computer architecture work. It loads BasicML instructions from a text file into a 100-word memory. From there, it runs them using an accumulator and supports input/output, loading and storing data, arithmetic, branching, and halting the program.

The application runs from the command line. The user provides a BasicML program file, and UVSim loads it into memory starting at location 00. The simulator then runs the instructions in order unless a branch instruction changes which one runs next. Any input or output the BasicML program needs happens through the console.

2. User Stories

User Story 1 – Load and Run a Program

As a computer science student, I want to load a BasicML program from a file and run it in UVSim so that I can see how machine-language instructions are executed.

User Story 2 – Work with BasicML Operations

As a computer science student, I want UVSim to perform input/output, memory, arithmetic, and control operations so that I can understand how a simple computer processes instructions and data.

3. Use Cases

UC-01: Load a BasicML Program

Actor: User

Description: The user provides the path to a BasicML text file. UVSim reads the file, validates the words, and loads the program into memory starting at location 00.

Preconditions: The program file exists and contains valid BasicML words.

Main Flow: The user starts UVSim and provides a file. UVSim reads the file and converts each BasicML word into a value. The program gets loaded into memory starting at location 00, and any unused memory is set to zero.

Expected Result: The program loads and UVSim is ready to run it. If the file is invalid, missing, or has more than 100 words, UVSim reports an error.

UC-02: READ Input

Actor: User

Description: UVSim executes a READ instruction with operation code 10.

Preconditions: A valid READ instruction is being executed.

Main Flow: UVSim prompts the user to enter a word in the console and stores it at the memory address given in the instruction.

Expected Result: A valid signed word is stored in the requested memory location. Invalid input should be rejected or reported as an error.

UC-03: WRITE Output

Actor: User

Description: UVSim executes a WRITE instruction with operation code 11.

Preconditions: The specified memory location contains a value.

Main Flow: UVSim reads the value stored at the instruction's memory address and displays it in the console.

Expected Result: The correct value is displayed to the user.

UC-04: LOAD a Value

Actor: UVSim

Description: UVSim executes a LOAD instruction with operation code 20.

Preconditions: The specified memory address is valid.

Main Flow: UVSim reads the word from the specified memory location and puts it in the accumulator.

Expected Result: The accumulator holds the value from that memory location.

UC-05: STORE a Value

Actor: UVSim

Description: UVSim executes a STORE instruction with operation code 21.

Preconditions: The accumulator contains a value and the destination memory address is valid.

Main Flow: UVSim copies the value currently in the accumulator to the specified memory location.

Expected Result: The destination memory location now holds the accumulator's value.

UC-06: ADD and SUBTRACT

Actor: UVSim

Description: UVSim performs addition or subtraction using operation codes 30 and 31.

Preconditions: The accumulator and specified memory location contain valid values.

Main Flow: For ADD, UVSim adds the value from the specified memory location to the accumulator. For SUBTRACT, UVSim subtracts the memory value from the accumulator.

Expected Result: The accumulator contains the result of the operation. A result outside the supported word range is treated as an error.

UC-07: MULTIPLY

Actor: UVSim

Description: UVSim executes a MULTIPLY instruction with operation code 33.

Preconditions: The accumulator and specified memory location contain valid values.

Main Flow: UVSim multiplies the accumulator by the value stored at the specified memory location.

Expected Result: The accumulator contains the result. A result outside the supported word range is treated as an error.

UC-08: DIVIDE

Actor: UVSIM

Description: UVSIM executes a DIVIDE instruction with operation code 32.

Preconditions: The accumulator contains a value and the specified memory location contains a valid divisor.

Main Flow: UVSIM divides the value in the accumulator by the value stored at the specified memory location.

Expected Result: The accumulator contains the result. Dividing by zero results in an error.

UC-09: Unconditional Branch

Actor: UVSIM

Description: UVSIM executes a BRANCH instruction with operation code 40.

Preconditions: The instruction contains a valid destination address.

Main Flow: UVSIM changes the instruction counter to the address specified by the BRANCH instruction.

Expected Result: Execution continues from the specified memory location instead of the next instruction in sequence.

UC-10: Conditional Branch

Actor: UVSIM

Description: UVSIM executes either BRANCHNEG (41) or BRANCHZERO (42).

Preconditions: A conditional branch instruction is being executed.

Main Flow: BRANCHNEG changes execution to the specified address when the accumulator is negative. BRANCHZERO changes execution to the specified address when the accumulator equals zero. If the condition is false, UVSIM continues to the next instruction.

Expected Result: UVSIM only branches when the accumulator meets the condition.

UC-11: HALT Program

Actor: UVSIM

Description: UVSIM executes a HALT instruction with operation code 43.

Preconditions: UVSIM is currently executing a BasicML program.

Main Flow: UVSIM recognizes the HALT instruction and stops running instructions.

Expected Result: Program execution ends normally.

UC-12: Handle Invalid Instructions and Program Errors

Actor: User / UVSIM

Description: UVSIM detects errors that keep a BasicML program from being loaded or run safely.

Preconditions: The program contains invalid input, an invalid instruction, or an operation that cannot be completed.

Main Flow: UVSIM catches the problem instead of carrying on with an invalid operation.

Examples include malformed BasicML words, programs larger than memory, invalid operation codes, arithmetic overflow, and division by zero.

Expected Result: UVSIM stops or rejects the operation and shows an appropriate error instead of continuing with wrong results.

4. High Level Program Flow

UVSIM starts by asking the user to provide a BasicML program file. It then reads and validates the file before loading the instructions into memory starting at location 00. Once the program is loaded, UVSIM steps through each instruction, reading its operation code and operand and performing the matching operation. The simulator keeps running instructions in order unless a branch instruction changes where execution goes next. This repeats until UVSIM hits a HALT instruction, which stops the program.